```
    d3.select("svg").selectAll("circle.median")
          .data(data)
          .enter()
          .append("circle")
          .attr("class", "tweets")
          .attr("r", 5)
          .attr("cx", function(d) {return xScale(d.day)})
          .attr("cy", function(d) {return yScale(d.median)})
          .style("fill", "darkgray");
}
```

But to get a better view of this data, we'll need to create a boxplot. Building a boxplot is similar to building a scatterplot, but instead of appending circles for each point of data, you append a `<g>` element. It's a good rule to always use `<g>` elements for your charts, because they allow you to apply labels or other important information to your graphical representations. But that means you'll need to use the `transform` attribute, which is how `<g>` elements are positioned on the canvas. Elements appended to a `<g>` base their coordinates off of the coordinates of their parent. When applying `x` and `y` attributes to child elements, you need to set them relative to the parent `<g>`.

Rather than selecting all the `<g>` elements and appending child elements one at a time, as we did in earlier chapters, we'll use the `.each()` function of a selection, which allows us to perform the same code on each element in a selection, to create the new elements. Like any D3 selection function, `.each()` allows you to access the bound data, array position, and DOM element. Earlier on, in chapter 1, we achieved the same functionality by using `selectAll` to select the `<g>` elements and directly append `<circle>`

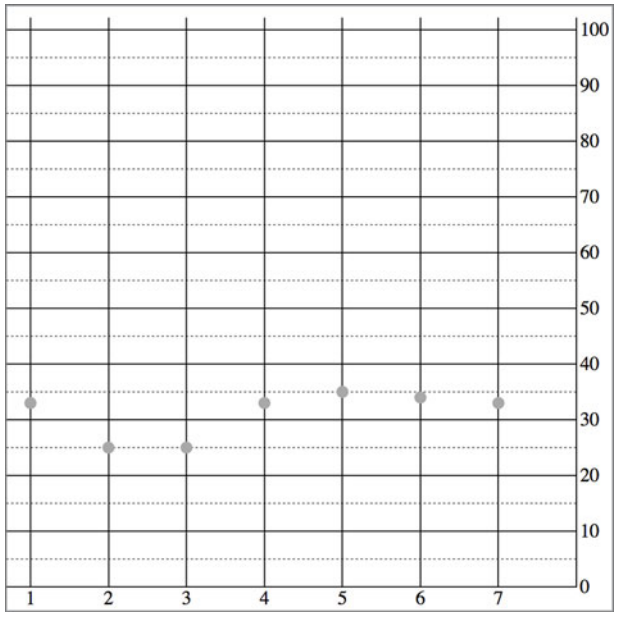


Figure 4.12 The median age of visitors (y-axis) by day of the week (x-axis) as represented by a scatterplot. It shows a slight dip in age on the second and third days.